

Event Objects

Event Objects is an event system that uses Unity's ScriptableObjects to broadcast events. Using a ScriptableObject offers numerous architectural and workflow benefits. In addition to the value of any event system – decoupling systems, improving modularity – working with ScriptableObjects is more designer-friendly. Unlike direct component references, Event Object references can be assigned in the Inspector, saved into prefabs, independent of other objects in the scene, or even across scenes. Event Objects also offer some debugging features uniquely possible using ScriptableObjects.

Benefits of a ScriptableObject-based Event System

- **Persistence Between Scenes:** ScriptableObject assets persist across scene loads, unlike MonoBehaviour instances which are tied to a GameObject in a specific scene. This makes Event Objects ideal for managing events or shared data that need to maintain state between different parts of the game without using a singleton class or "DontDestroyOnLoad" object.
- **Designer-Friendly Workflow:** Creating Event Objects in the Project panel can be assigned in the Inspector. This allows non-technical team members, such as designers, to hook up game logic and interactions (like triggering sound effects or UI changes) without writing or changing C# code.
- **Easier Testing and Debugging:** At runtime, events can be manually triggered directly from an Event Object Inspector in the Editor, allowing for isolated testing of how different systems respond to specific events without having to play through the entire game flow. Event Objects Inspectors also display which objects are listening and which are invoking events, aiding in tracing the flow of logic.
- **More Efficient:** Event Objects are self-contained and do not rely on a Singleton event hub. This pattern provides a superior, more flexible alternative to the Singleton pattern for globally accessible functionality or data, avoiding common drawbacks like rigid behavior and difficulty in testing.

Event Object Types

- **Gauge:** Gauges contain values that need to be globally accessed, for example, the player's score. Any component that needs to be informed when the player's score changes can add a callback to the Gauge that will be invoked whenever the score changes. Gauges can optionally be made persistent—meaning that their value will be

automatically saved in PlayerPrefs and reloaded whenever the game is started. This is particularly useful for things like game settings.

- **Party Line:** PartyLines facilitate many-to-many communication. Unlike Messenger and Mailbox, a Party Line can have multiple senders and multiple receivers of messages.
- **Messenger:** Messenger restricts events to a one-to-many relationship, where only one component is permitted to send events, while many can receive them. This is useful to enforce intended architectural flow.
- **Mailbox:** Mailbox is designed for a many-to-one relationship. It can receive messages from multiple senders, but its contents may only be consumed by a single receiver. The component that receives messages from a Mailbox can receive individual messages immediately, or allow messages to collect and be processed periodically en masse. It can also, optionally, send a response to the sender of the message.

Keys

Changing Gauge values or sending events through the other Event Object types is done using a “Key” object. With a reference to the Event Object, the key can be created – and should then be cached until no longer required, when it should be disposed. To create a key, a component “owner” must be passed in; likewise, when subscribing to receive events, the subscribing component must be passed in. At runtime, the Inspectors for all Event Objects show what components own keys and which are subscribed to receive those events – which can be very useful for debugging purposes.

Working with Gauges

To store the Player’s score, we would create a new Gauge that works with integer values. Right-click in the Project panel and then **Create » Barliesque » Event Objects » Gauges » int** and give the new asset a name like PlayerScore. Add a serialized field to whatever component needs access to the player’s score:

```
[SerializeField] private GaugeInt _playerScore;
```

Now, for a component that needs to manage the player’s score, we need a key:

```
private Gauge<int>.IKey _playerScoreKey;
```

```
private void Start()
{
    _playerScoreKey = _playerScore.CreateKey(this, null);
}
```

When creating a key, the first parameter is a reference to the component owner of this new key. The second parameter is for a change handler. This change handler will only be called whenever **another component** changes the Gauge. When this key is used to change the Gauge value, the change handler is not called.

Now that we have a key, we can update the value of the key, automatically sending update events to any components subscribed for those events.

```
private void PlayerHitTarget()
{
    // Award 100 points
    _playerScoreKey.Value += 100;
}

private void OnDestroy()
{
    _playerScoreKey.Dispose();
}
```

When we're done with the key, it's important to call `Dispose()`.

Now, for a component that needs to receive events of the Gauge's value being changed, we'll add a watcher:

```
[SerializeField] private GaugeInt _playerScore;

private void Start()
{
    _playerScore.AddWatcher(this, UpdateScoreboard);
}

private void UpdateScoreboard(int score)
{
}
```

```

        _scoreboard.SetText($"Score: {score} pts");
    }

    private void OnDestroy()
    {
        _playerScore.RemoveWatcher(this, UpdateScoreboard);
    }

```

Custom Gauge Types

The base Gauge type is designed to be implemented for any value type. Most common types are already implemented, but suppose you needed a double precision float Gauge? Of course, float is already supported, but double is not. So, a new class would be needed that serializes and deserializes the value, as well as registering the menu option to create this new Gauge type:

```

[CreateAssetMenu(fileName = "Gauge(double)",
    menuName = "Barliesque/Event Objects/Gauges/double", order = 1)]
public class GaugeDouble : Gauge<double>, Gauge<double>.ISerializable
{
    public string Serialize(double value)
    {
        return value.ToString();
    }

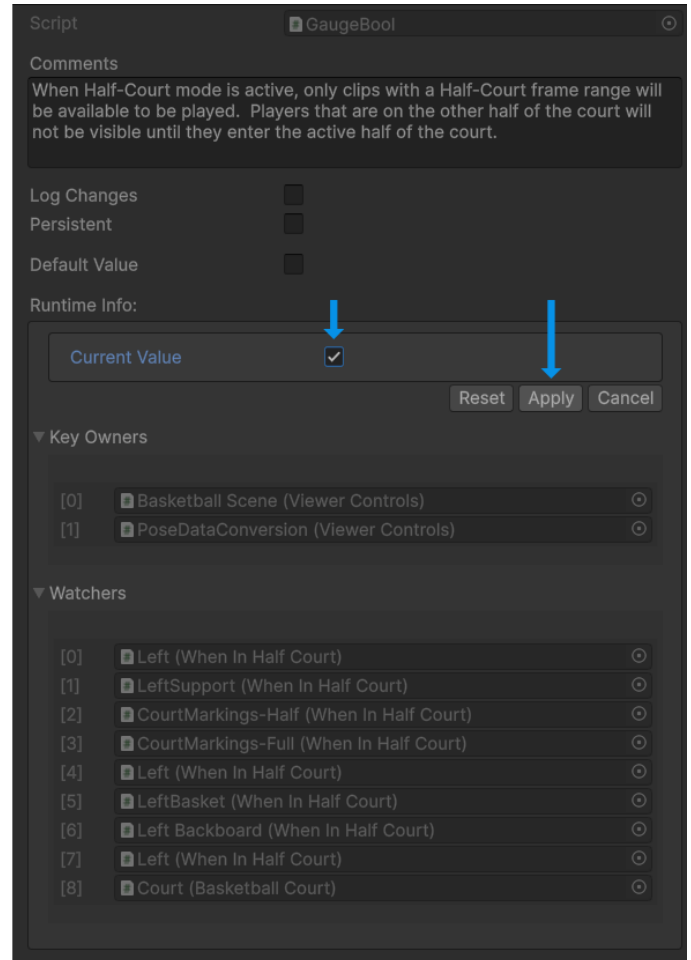
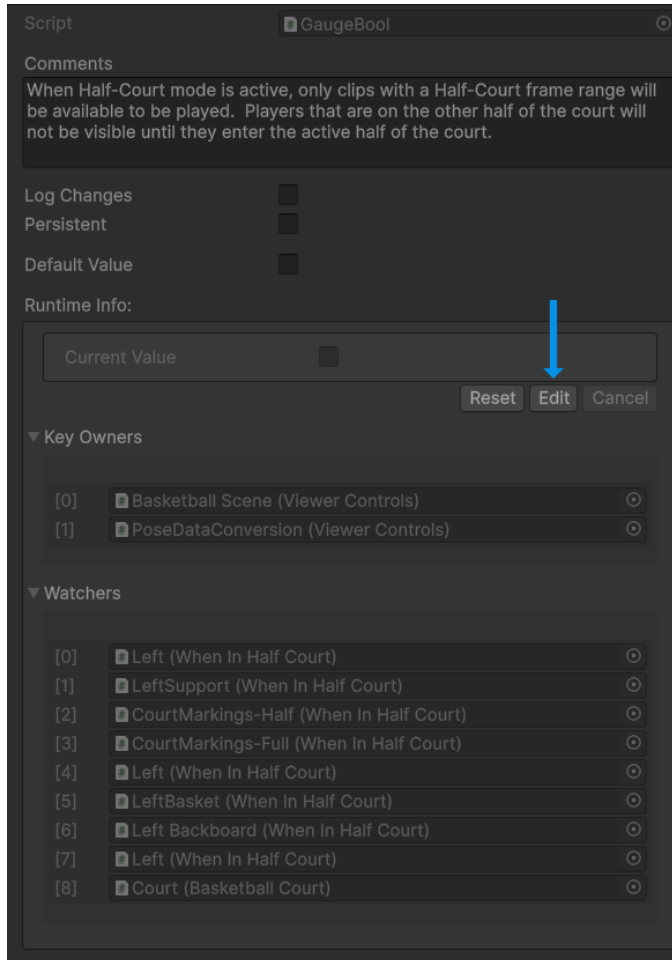
    public double Deserialize(double serial)
    {
        return double.Parse(serial);
    }
}

```

Any value type that the Inspector is able to work with can be supported with no more than the above implementation. Unfortunately, the higher precision decimal type is not supported by the Unity Inspector. Although a GaugeDecimal implementation would function, the Inspector for that Gauge would not. In this and in the case of custom value types, a CustomPropertyDrawer would need to be implemented as well. See TimePeriodDrawer.cs for an example of this.

Gauge Debug Features

At runtime, the Gauge inspector offers debugging advantages not found in most event systems. We can see all key owners, as well as watchers of the Gauge. Click on any of these to ping the GameObject on which they reside to easily locate all components with direct links to the Gauge.



On the left, you can see the current value of the Gauge is visible, as well as a mechanism to change its value. When the value is changed via the Inspector, the same event is invoked exactly as if any key owner component in the project had changed the value, invoking each of those Watchers, and allowing for easy testing.

Working with PartyLine

A `PartyLine` allows any number of entities to send or receive messages. Unlike `Gauge`, the type of the message is not determined by the Event Object asset. All messages sent via `PartyLine` must be of the same type. To maintain consistently typed messages, the first time a key is created, the `PartyLine` sets the value type used; any subsequent attempt to create a key of a different type will cause an error to be thrown.

```
[SerializeField] private PartyLine _partyLine;

private PartyLine.IKey<float> _partyLineKey;

private void Awake()
{
    _partyLineKey = _partyLine.CreateKey<float>(this, SomethingHappened);
}
```

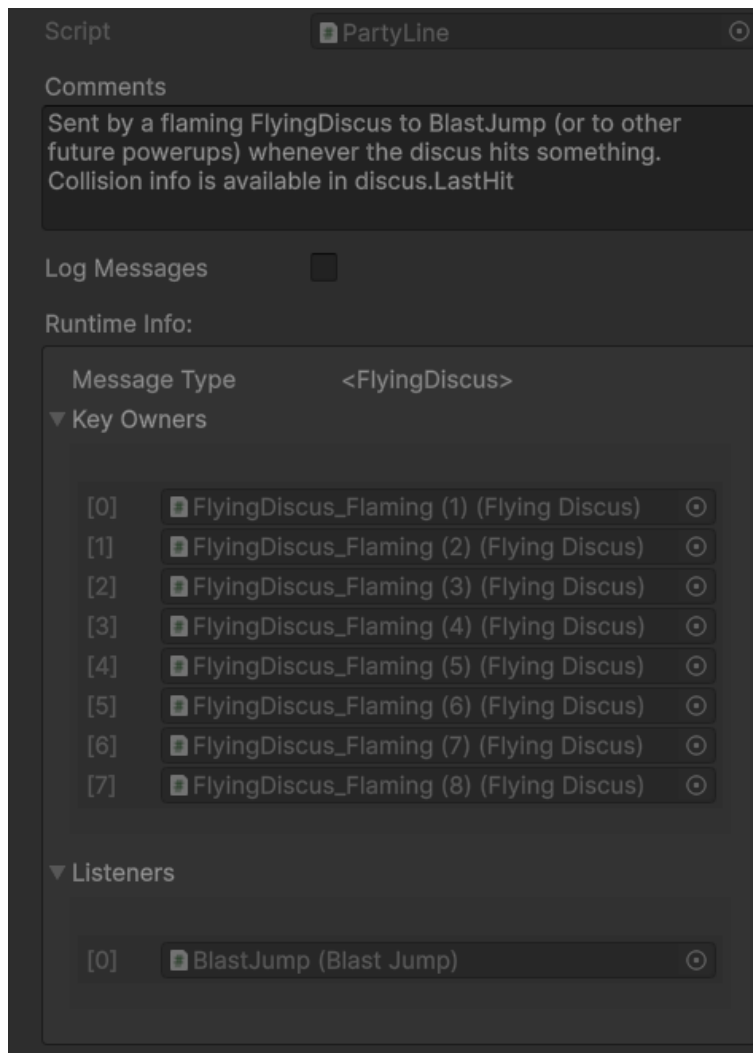
Some components may only need to listen for messages without needing to send messages. This can be done without a key, directly from the `PartyLine` object:

```
[SerializeField] private PartyLine _partyLine;

private void Start()
{
    _partyLine.AddListener<float>(this, SomethingHappened);
}
```

So, a component that creates a Key has two options for receiving events from the `PartyLine`: by calling `AddListener()` or by passing in a handler when creating the Key, as shown above. `AddListener()` will set a callback that's invoked whenever **any component** sends a message across the `PartyLine`. However, the handler set when creating a key **will not be invoked** when the message is sent **from this key**.

At runtime, the message type, once established, is shown in the Event Object's Inspector. Here's an example from another project:



All EventObjects include a Comments field. Use this to document the purpose of the PartyLine.

[Read about how PartyLine got its name.](#)

Working with Messenger

`Messenger` EventObjects allow a one-to-many relationship: one sender, multiple receivers of each message. Use `Messenger` to enforce architectural choices where a particular component is “in charge” of sending those messages.

Messengers have the option to send data to their subscribers. The type of message data sent with a `Messenger` is set when the key is created. Once set, the type cannot be changed, since

this would cause conflicts among receivers of messages. To create a key to send message data, use the generic version of `CreateKey()`.

```
[SerializeField] private Messenger _playerInjured;

private Messenger.IKey<int> _playerInjuredKey;

private void Start()
{
    _playerInjuredKey = _playerInjured.CreateKey<int>(this);
}
```

If a second component attempts to create a key, an error will be thrown to ensure the `Messenger` is used as intended.

A `Messenger` can require a “response” value from its Subscribers. In this example, the `Messenger` is set up to broadcast an integer value, and receive a `bool` in return.

```
[SerializeField] private Messenger _playerInjured;

private Messenger.IKey<int, bool> _playerInjuredKey;

private void Start()
{
    _playerInjuredKey = _playerInjured.CreateKey<int, bool>(this);
}

private void ApplyInjury(int intensity)
{
    _playerInjuredKey.SendMessage(intensity, IsPlayerStillAlive);
}

private void IsPlayerStillAlive(bool living)
{
    if (living) return;
    PlayerDied();
}
```

After each subscriber returns a value, the response handler (`IsPlayerStillAlive()` in the above example) is called. Message and Response types will appear in the Messenger Inspector at runtime for reference.

Working with Mailbox

The `Mailbox` `EventObject` implements a many-to-one flow of event messages. No key is needed to send a message. Only one Key can exist at a time, to enable receiving messages from the `Mailbox`. The Key can be set up to automatically call a handler whenever a message is sent:

<< Example of an immediate handler >>

...or the Mailbox can hold messages until the Key owner is ready to receive them:

<< Example of key set up to hold messages, and a function that gets held messages >>

The event handler can optionally return data to the message sender—in which case, the sender will need to supply a response handler callback:

<< Example of the key owner sending response data >>

<< Example of a message sent with a response handler >>